Software Testing Strategies and Methods

CONTENTS

I. Software Testing Fundamentals
    1. Definition of Software Testing
II. Characteristics of Testing Strategies
III. Software Verification and Validation (V&V)
IV. Testing Strategies
    1. Unit Testing
    2. Integration Testing
V. Alpha and Beta Testing (Concept and differences)
VI. System Testing
    1. Concept of System Testing
VII. Concept of White-box and Black-Box Testing
VIII. Debugging
    1. Concept and need of Debugging
    2. Characteristics of bugs
IX. Debugging Strategies
    1. Concept of Brute Force, Back Tracking, Induction, Deduction

I. Software Testing Fundamentals

1. Definition of Software Testing

i. Software testing is a method of assessing the functionality of a software program.

ii. There are many different types of software testing but the two main categories are dynamic testing and static testing.

II. Characteristics of Testing Strategies

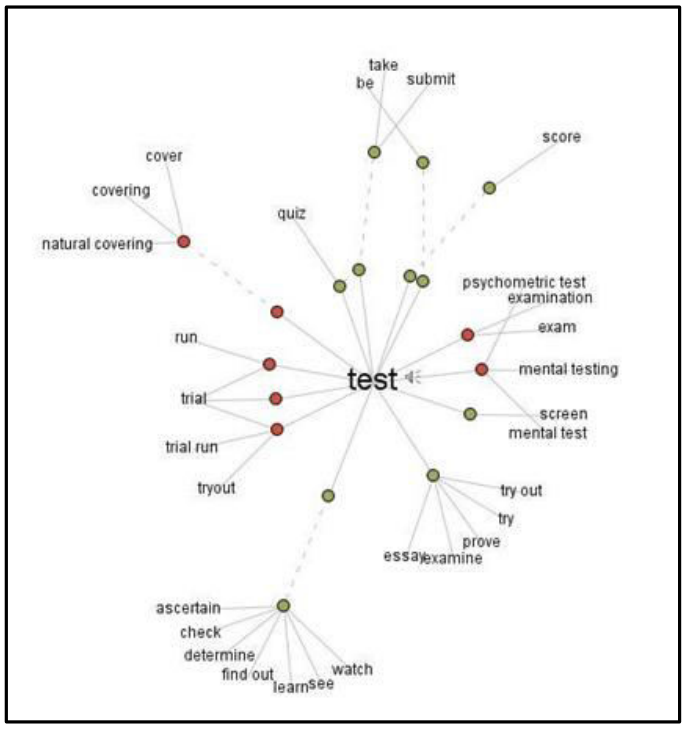


Figure 1: Software Testing

Software Testing Myths and Facts: Just as every field has its myths, so does the field of Software Testing. Software testing myths have arisen primarily due to the following:

i. Lack of authoritative facts.

ii. Evolving nature of the industry.

iii. General flaws in human logic.

Some of the myths are explained below, along with their related facts:

1. **MYTH**: Quality Control = Testing.
   - **FACT**: Testing is just one component of software quality control. Quality Control includes other activities such as Reviews.

2. **MYTH**: The objective of Testing is to ensure a 100% defect- free product.
    - **FACT**: The objective of testing is to uncover as many defects as possible. Identifying all defects and getting rid of them is impossible.
3. **MYTH**: Testing is easy.
    - **FACT**: Testing can be difficult and challenging (sometimes, even more so than coding).
4. **MYTH**: Anyone can test.
    - **FACT**: Testing is a rigorous discipline and requires many kinds of skills.
5. **MYTH**: There is no creativity in testing.
    - **FACT**: Creativity can be applied when formulating test approaches, when designing tests, and even when executing tests.
6. **MYTH**: Automated testing eliminates the need for manual testing.
    - **FACT**: 100% test automation cannot be achieved. Manual Testing, to some level, is always necessary.
7. **MYTH**: When a defect slips, it is the fault of the Testers.
    - **FACT**: Quality is the responsibility of all members/stakeholders, including developers, of a project.
8. **MYTH**: Software Testing does not offer opportunities for career growth.
    - **FACT**: Gone are the days when users had to accept whatever product was dished to them; no matter what the quality.
    - With the abundance of competing software and increasingly demanding users, the need for software testers to ensure high quality will continue to grow.

III. Software Verification and Validation (V&V)

(Question: Differentiate between verification and validation- 6 Marks)

Criteria	**Verification**	**Validation**
Definition	The process of evaluating work-products (not the actual final product) of a development phase to determine whether they meet the specified requirements for that phase.	The process of evaluating software during or at the end of the development process to determine whether it satisfies specified business requirements.
Objective	To ensure that the product is being built according to the requirements and design specifications. In other words, to ensure that work products meet their specified requirements.	To ensure that the product actually meets the user's needs, and that the specifications were correct in the first place. In other words, to demonstrate that the product fulfills its intended use

		when placed in its intended environment.
Question	Are we building the product *right*?	Are we building the *right* product?
Evaluation Items	Plans, Requirement Specs, Design Specs, Code, Test Cases	The actual product/software.
Activities	▪ Reviews ▪ Walkthroughs ▪ Inspections	▪ Testing

IV. Testing Strategies

(Question: Explain the software testing strategies – 8 Marks)

1. Unit Testing

(Question: Explain unit testing with Diagram- 4 marks)

i. **Unit Testing** is a level of the software testing process where individual units/components of a software/system are tested.

ii. The purpose is to validate that each unit of the software performs as designed.

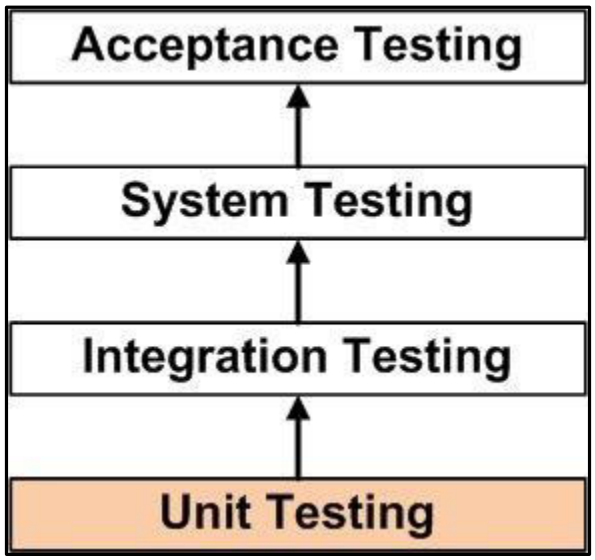


Figure 2: Unit Testing

iii. A unit is the smallest testable part of software.

iv. It usually has one or a few inputs and usually a single output.

v. In procedural programming a unit may be an individual program, function, procedure, etc.

vi. In object-oriented programming, the smallest unit is a method, which may belong to a base/super class, abstract class or derived/child class.

Advantages

i. Unit testing increases confidence in changing/maintaining code.
ii. If good unit tests are written and if they are run every time any code is changed, the likelihood of any defects due to the change being promptly caught is very high.
iii. If unit testing is not in place, the most one can do is hope for the best and wait till the test results at higher levels of testing are out.
iv. If codes are already made less interdependent to make unit testing possible, the unintended impact of changes to any code is less.
v. Codes are more reusable. In order to make unit testing possible, codes need to be modular. This means that codes are easier to reuse.
vi. The cost of fixing a defect detected during unit testing is lesser in comparison to that of defects detected at higher levels.
vii. Compare the cost (time, effort, destruction, humiliation) of a defect detected during acceptance testing or say when the software is live.
viii. Debugging is easy. When a test fails, only the latest changes need to be debugged. With testing at higher levels, changes made over the span of several days/weeks/months need to be debugged.

2. Integration Testing

(Question: Explain integration testing with diagram and give its advantages and disadvantages- 6 Marks)

i. **Integration Testing** is a level of the software testing process where individual units are combined and tested as a group.

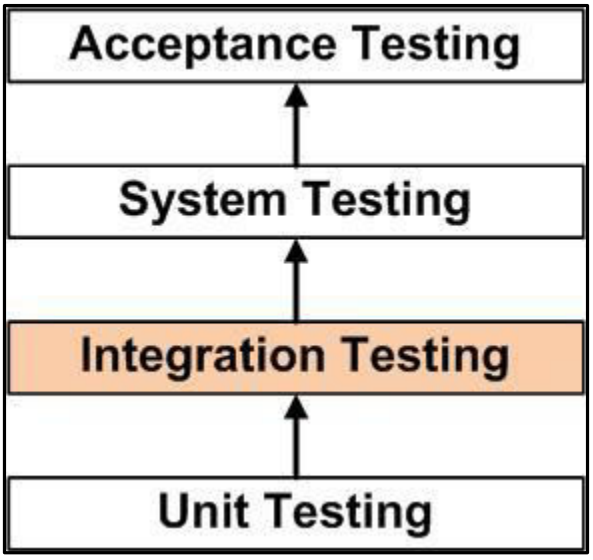


Figure 3: Integration Testing

ii. The purpose of this level of testing is to expose faults in the interaction between integrated units.
iii. Test drivers and test stubs are used to assist in Integration Testing.

V. Alpha and Beta Testing

1. Alpha Testing

(Question: Explain alpha testing – 4 Marks)

i. Alpha testing is one of the most common software testing strategy used in software development. It's specially used by product development organizations.
ii. This **test takes place at the developer's site**. Developers observe the users and note problems.
iii. Alpha testing is testing of an application when development is about to complete. Minor design changes can still be made as a result of alpha testing.
iv. Alpha testing is typically performed by a group that is independent of the design team, but still within the company, e.g. in-house software test engineers, or software QA engineers.
v. Alpha testing is final testing before the software is released to the general public. It has two phases:
   a. In the **first phase** of alpha testing, the software is tested by in-house developers. They use either debugger software, or hardware-assisted debuggers. The goal is to catch bugs quickly.
   b. In the **second phase** of alpha testing, the software is handed over to the software QA staff, for additional testing in an environment that is similar to the intended use.
vi. Alpha testing is simulated or actual operational testing by potential users/customers or an independent test team at the developers' site.
vii. Alpha testing is often employed for off-the-shelf software as a form of internal acceptance testing, before the software goes to beta testing.

2. Beta testing

(Question: Explain beta testing. – 4 Marks)

i. It is also known as field testing.
ii. It takes place at **customer's site**.
iii. It sends the system to users who install it and use it under real-world working conditions.

iv. A beta test is the second phase of software testing in which a sampling of the intended audience tries the product out.
v. Beta testing can be considered "pre-release testing.
vi. The goal of beta testing is to place your application in the hands of real users outside of your own engineering team to discover any flaws or issues from the user's perspective that you would not want to have in your final, released version of the application.
vii. Closed beta versions are released to a select group of individuals for a user test and are invitation only, while
viii. Open betas are from a larger group to the general public and anyone interested.
ix. The testers report any bugs that they find, and sometimes suggest additional features they think should be available in the final version.

VI. System Testing

1. Concept of System Testing

i. In system testing the behavior of whole system/product is tested as defined by the scope of the development project or product.
ii. It may include tests based on risks and/or requirement specifications, business process, use cases, or other high level descriptions of system behavior, interactions with the operating systems, and system resources.
iii. System testing is most often the final test to verify that the system to be delivered meets the specification and its purpose.
iv. System testing is carried out by specialist's testers or independent testers.
v. System testing should investigate both functional and non-functional requirements of the testing.

VII. Concept of White-box and Black-Box Testing

(Question: Differentiate between black box testing and white box testing - 6 Marks)

1. Black Box Testing

i. **Black Box Testing**, also known as Behavioral Testing, is a software testing method in which the internal structure/design/implementation of the item being tested is not known to the tester.
ii. These tests can be functional or non-functional, though usually functional.

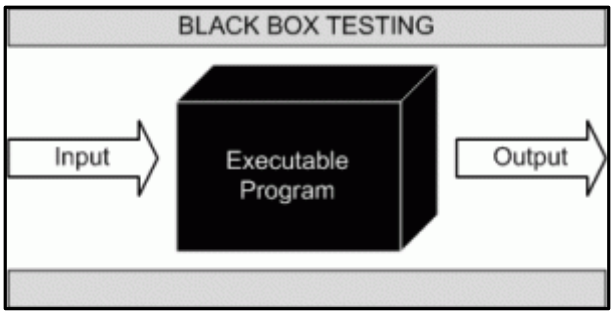


Figure 4: Black Box Testing

iii. This method is named so because the software program, in the eyes of the tester, is like a black box; inside which one cannot see.

iv. Black Box Testing is contrasted with White Box Testing. View Differences between Black Box Testing and White Box Testing.

v. This method of attempts to find errors in the following categories:
   - Incorrect or missing functions
   - Interface errors
   - Errors in data structures or external database access
   - Behavior or performance errors
   - Initialization and termination errors

Black Box Testing Techniques

(Question: Explain the various black box testing techniques- 6 Marks)

i. *Equivalence partitioning*

a) Equivalence Partitioning is a software test design technique that involves dividing input values into valid and invalid partitions and selecting representative values from each partition as test data.

ii. Boundary Value Analysis

a) Boundary Value Analysis is a software test design technique that involves determination of boundaries for input values and selecting values that are at the boundaries and just inside/outside of the boundaries as test data.

iii. Cause Effect Graphing

a) Cause Effect Graphing is a software test design technique that involves identifying the cases (input conditions) and effects (output conditions), producing a Cause-Effect Graph, and generating test cases accordingly.

BLACK BOX TESTING ADVANTAGES

i. Tests are done from a user's point of view and will help in exposing discrepancies in the specifications

ii. Tester need not know programming languages or how the software has been implemented
iii. Tests can be conducted by a body independent from the developers, allowing for an objective perspective and the avoidance of developer-bias
iv. Test cases can be designed as soon as the specifications are complete

BLACK BOX TESTING DISADVANTAGES

i. Only a small number of possible inputs can be tested and many program paths will be left untested
ii. Without clear specifications, which is the situation in many projects, test cases will be difficult to design
iii. Tests can be redundant if the software designer/ developer has already run a test case.
iv. Ever wondered why a soothsayer closes the eyes when foretelling events? So is almost the case in Black Box Testing.

2. White Box Testing

i. **White Box Testing** (also known as Clear Box Testing, Open Box Testing, Glass Box Testing, Transparent Box Testing, Code-Based Testing or Structural Testing) is a software testing method in which the internal structure/design/implementation of the item being tested is known to the tester.
ii. The tester chooses inputs to exercise paths through the code and determines the appropriate outputs.
iii. Programming know-how and the implementation knowledge is essential.
iv. White box testing is testing beyond the user interface and into the nitty-gritty of a system.

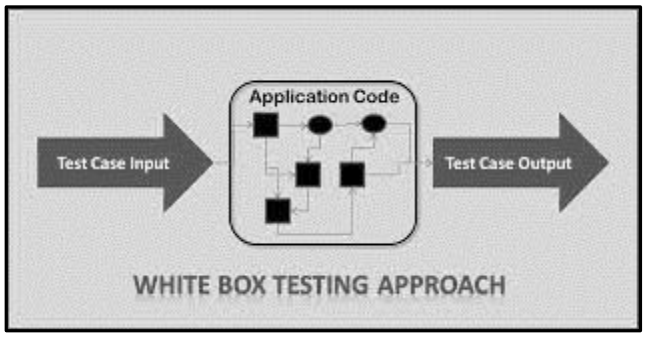


Figure 5: White Box Testing

White Box Testing Advantages

i. Testing can be commenced at an earlier stage. One need not wait for the GUI to be available.
ii. Testing is more thorough, with the possibility of covering most paths.

White Box Testing Disadvantages

i. Since tests can be very complex, highly skilled resources are required, with thorough knowledge of programming and implementation.
ii. Test script maintenance can be a burden if the implementation changes too frequently.
iii. Since this method of testing it closely tied with the application being testing, tools to cater to every kind of implementation/platform may not be readily available.
iv. White Box Testing is like the work of a mechanic who examines the engine to see why the car is not moving.

Criteria	Black Box Testing	White Box Testing
Definition	Black Box Testing is a software testing method in which the internal structure/ design/ implementation of the item being tested is NOT known to the tester	White Box Testing is a software testing method in which the internal structure/ design/ implementation of the item being tested is known to the tester.
Levels Applicable To	Mainly applicable to higher levels of testing: Acceptance Testing System Testing	Mainly applicable to lower levels of testing: Unit Testing Integration Testing
Responsibility	Generally, independent Software Testers	Generally, Software Developers
Programming Knowledge	Not Required	Required
Implementation Knowledge	Not Required	Required
Basis for Test Cases	Requirement Specifications	Detail Design

VIII. Debugging

1. Concept and need of Debugging

(Question: Describe the concept and need of debugging in software testing- 6 Marks)

i. Debugging is the process of locating and fixing or bypassing bugs (errors) in computer program code or the engineering of a hardware device.

ii. To *debug* a program or hardware device is to start with a problem, isolate the source of the problem, and then fix it.
iii. A user of a program that does not know how to fix the problem may learn enough about the problem to be able to avoid it until it is permanently fixed.
iv. When someone says they've debugged a program or "worked the bugs out" of a program, they imply that they fixed it so that the bugs no longer exist.
v. Debugging is a necessary process in almost any new software or hardware development process, whether a commercial product or an enterprise or personal application program.
vi. For complex products, debugging is done as the result of the unit test for the smallest unit of a system, again at component test when parts are brought together, again at system test when the product is used with other existing products, and again during customer beta test, when users try the product out in a real world situation.
vii. Because most computer programs and many programmed hardware devices contain thousands of lines of code, almost any new product is likely to contain a few bugs.
viii. Invariably, the bugs in the functions that get most use are found and fixed first. An early version of a program that has lots of bugs is referred to as "buggy."
ix. Debugging tools (called debuggers) help identify coding errors at various development stages. Some programming language packages include a facility for checking the code for errors as it is being written.

2. Characteristics of bugs

(Question: Explain the various characteristics of bugs – 4 Marks)

i. A defect is an error or a bug, in the application which is created.
ii. A programmer while designing and building the software can make mistakes or error.
iii. These mistakes or errors mean that there are flaws in the software. These are called defects.
iv. When actual result deviates from the expected result while testing a software application or product then it results into a defect.

v. Hence, any deviation from the specification mentioned in the product functional specification document is a defect. In different organizations it's called differently like bug, issue, incidents or problem.
vi. When the result of the software application or product does not meet with the end user expectations or the software requirements then it results into a Bug or Defect.
vii. These defects or bugs occur because of an error in logic or in coding which results into the failure or unpredicted or unanticipated results.

IX. Debugging Strategies

(Question: Explain any two debugging strategies- 8 marks)

1. Brute Force

i. This method is most common and least efficient for isolating the cause of a software error.
ii. We apply this method when all else fail. In this method, a printout of all registers and relevant memory locations is obtained and studied.
iii. All dumps should be well documented and retained for possible use on subsequent problems.

2. Back Tracking

i. It is a quite popular approach of debugging which is used effectively in case of small applications.
ii. The process starts from the site where a particular symptom gets detected, from there on backward tracing is done across the entire source code till we are able to lay our hands on the site being the cause.
iii. As the number of source lines increases, the number of potential backward paths may become unmanageably large.

3. Cause Elimination(Induction and Deduction)

i. The third approach to debugging, because elimination, is manifested by induction or deduction and introduces the concept of binary partitioning. This approach is also called induction and deduction.
ii. Data related to the error occurrence are organized to isolate potential causes.
iii. A "cause hypothesis" is devised and the data are used to prove or disprove the hypothesis.
iv. A list of all possible causes is developed and tests are conducted to eliminate each.

v. If initial tests indicate that a particular cause hypothesis shows promise, the data are refined in an attempt to isolate the bug.